

Guía rápida · GenericDataTable (React + DataTables)

Qué es

Componente de tabla genérica sobre DataTables (jQuery) con columnas dinámicas, exportación a Excel, render por celda, y dos modos: controlado por props o independiente con API imperativa.

Uso mínimo (ejemplo)

```
<GenericDataTable<DTO_Cuenta>
  ref={tableRef}
  title="Cuentas"
  columnKeys={columnKeysCuenta}
  labelMap={labelMapCuenta}
  data={accountsPayable}           // se carga 1 sola vez si 'independent'
  independent                       // no escucha cambios del padre después de cargar
  idField="id_Cuenta"              // campo ID para upsert/remove
  onAdd={handleAddNew}
  onEdit={handleEdit}
  onDelete={handleDelete}
  disableButtonAdd={disableButtonAdd}
  includeEstadoColumn={false}
  customRenderers={customRenderers}
  customColumns=[[detalleJSONColumn]]
  dataTableButtons={dataTableButtons}
  onRowClick={(row) => { setRowTableSelected(row); setIsInfoModalOpen(true); }}
/>
```

Nota: Los valores mostrados son solo EJEMPLOS. Puedes cambiarlos según tu DTO y tus necesidades.

Parámetros clave (resumen)

- **title:** Texto del encabezado de la tarjeta.
- **columnKeys:** Claves del objeto T que serán columnas (ordenadas).
- **labelMap:** Mapa clave → etiqueta; con 'avance' agrega barra de progreso; con 'estado' + includeEstadoColumn agrega badge.
- **data:** Dataset inicial. En independiente se carga una sola vez; en controlado refresca en cada cambio.
- **onAdd/onEdit/onDelete:** Callbacks de acciones (botón Agregar y acciones por fila).
- **customRenderers:** Render React por celda para claves específicas.
- **includeEstadoColumn:** Si true y labelMap['estado'], añade columna Estado.
- **customColumns:** Columnas extra (por ejemplo, 'detalleJSONColumn').
- **dataTableButtons:** Botones extra por fila (además de Editar/Eliminar).
- **nowrapColumns:** Fuerza 'text-nowrap' por clave o título.
- **onRowClick:** Click en fila (no primera/última ni filas child).

- **independent:** Si true, usa API imperativa (load/upsert/remove...).
- **idField:** Nombre del campo ID para upsert/remove (p. ej. 'iD_Cuenta').

Ejemplos de parámetros (personalizables)

Estos ejemplos funcionan tal cual, pero puedes cambiarlos para cualquier DTO que uses con la tabla.

1) *columnKeysCuenta*

```
export const columnKeysCuenta: (keyof DTO_Cuenta)[] = [
  "iD_Cuenta",
  "monto",
  "montoAbonado",
  "saldoPendiente",
  "estadoPago",
  "tipoCuenta",
  "fechaLimite",
  "fechaInicial",
  "concepto",
];
```

- Representa el orden y las claves del DTO a mostrar como columnas. Puedes agregar/quitar/reordenar según tu modelo.

2) *labelMapCuenta*

```
export const labelMapCuenta: Record<string, string> = {
  iD_Cuenta: "ID",
  iD_OrdenServicio: "Orden",
  estado: "Estado",
  concepto: "Concepto",
  monto: "Monto",
  fechaInicial: "Fecha Inicial",
  fechaLimite: "Fecha Límite",
  tipoCuenta: "Tipo De Cuenta",
  fechaModificacion: "Fecha de Modificación",
  detalleJSON: "Detalles",
  montoAbonado: "Abonado",
  saldoPendiente: "Saldo",
  estadoPago: "Estado",
};
```

- Define las etiquetas visibles por columna. Si incluyes 'avance' en este map, la tabla agrega una columna de barra de progreso; si incluyes 'estado' y 'includeEstadoColumn' es true, agrega la columna Estado.

3) *Estado 'data' tipado (accountsPayable)*

```
const [accountsPayable, setAccountsPayable] = useState<DTO_Cuenta[]>([]);

// Se pasa como 'data' en la tabla (carga inicial one-shot en 'independent'):
<GenericDataTable<DTO_Cuenta> data={accountsPayable} independent ... />
```

- En modo independiente, 'data' se inyecta una vez al montar. Luego usa la API imperativa para mutar filas sin depender del estado del padre.

4) DTO_Cuenta (ejemplo de modelo)

```
export class DTO_Cuenta {
  id_Cuenta: number = 0;
  id_Negocio: number = 0;
  id_OrdenServicio?: number;
  estado: DTO_Estado = new DTO_Estado();
  concepto: string = "";
  descripcion?: string;
  monto: number = 0;
  tipoCuenta: string = "";
  detalleJSON?: DTO_DetalleCuentaJSON;
  fechaInicial: Date = new Date();
  fechaModificacion: Date = new Date();
  fechaLimite?: Date;

  // Campos calculados
  montoAbonado: number = 0;
  saldoPendiente: number = 0;
  estadoPago?: string;
}
```

- Es solo un ejemplo. La tabla funciona con cualquier T. Asegúrate de pasar 'idField' con el nombre correcto (por ejemplo, 'id_Cuenta').

5) Columna personalizada (customColumns): detalleJSONColumn

```
const detalleJSONColumn = {
  title: labelMap["detalleJSON"] ?? "Detalle",
  data: "detalleJSON",
  orderable: true,
  searchable: true,
  className: "text-center",
  defaultContent: "",
  render: function (_, unknown, type: "display" | "export" | "filter" | "sort", row: DTO_Cuenta) {
    return formatDetalleJSON(row?.detalleJSON ?? {}, type);
  },
};
// Se pasa así: customColumns=[detalleJSONColumn]
```

- Te permite añadir columnas no listadas en 'columnKeys'. Usa 'render' y soporta 'type' para export/filter/sort.

6) Renderer de detalle (formatDetalleJSON)

```
export function formatDetalleJSON(
  detalle: any,
  type: "display" | "export" | "filter" | "sort" = "display"
): string {
  const filas = detalle?.filas ?? [];
  const desc = detalle?.descuento?.valor ?? "0";
```

```

const tipoDesc = detalle?.descuento?.nombre ?? "Monto";
const imp = detalle?.impuesto?.valor ?? "0";

const descStr = tipoDesc === "Monto"
  ? `Desc: ■${Number(desc).toLocaleString("es-CR",{ minimumFractionDigits: 2 })}`
  : `Desc: ${Number(desc)}%`;

const impStr = `Imp: ${Number(imp)}%`;
let filasStr = filas.length === 0 ? "0"
  : filas.map((f: any) => `${f.nombre}: ■${Number(f.valor).toLocaleString("es-CR",{ minimumFractionDigits: 2 })}`);

if (type === "export" || type === "filter" || type === "sort") {
  return `${descStr}, ${impStr}, ${filasStr}`;
}

return `
<div class="row dt-hover-invert">
  <div class="col-auto">
    <span class="d-inline-block text-truncate bg-primary-subtle text-dark px-2 py-1 w-100" style="display: inline-block; width: 100%; height: 1.5em; vertical-align: middle;">

```

- Para export/filter/sort devuelve texto plano; para display devuelve HTML con estilos.

7) Botones extra por fila (dataTableButtons)

```

const dataTableButtons: DynamicButtonConfig[] = [
  {
    titulo: "Ver Transacciones",
    icon: <i className="bi bi-arrow-left-right fs-5 me-1" />,
    onClick: (row) => { handleTransaction(row); },
  },
];
// Se pasan así: dataTableButtons={dataTableButtons}

```

- Añade acciones personalizadas junto a Editar/Eliminar. El 'row' es la fila completa.

API imperativa (modo independiente)

```

// Preparación
const tableRef = useRef<GenericDataTableHandle<DTO_Cuenta>>(null);

// Insertar o actualizar (sube al tope)
tableRef.current?.upsert(cuenta);

// Insertar/actualizar varias
tableRef.current?.bulkUpsert(cuentas);

```

```
// Eliminar por ID
tableRef.current?.removeById(cuenta.id_Cuenta);

// Cargar listado completo (reemplaza todo)
tableRef.current?.load(listado);

// Limpiar todo
tableRef.current?.clear();

// Obtener snapshot actual
const rows: DTO_Cuenta[] = tableRef.current?.getData() ?? [];
```

Consejos rápidos

- Asegura que 'idField' coincida con tu clave primaria real (p. ej. 'ID_Cuenta').
- En 'independent', no necesitas volver a setear 'data' para ver cambios; usa la API (upsert/remove/load).
- Si llamas a la API antes de que la tabla inicie, la operación se encola y se ejecuta sola al terminar la inicialización.
- Puedes adaptar fácilmente los ejemplos para cualquier otro DTO (solo cambia T, columnKeys, labelMap e idField).